Let's walk through the text to embedding conversion process:

1. We start by testing out our chosen model on a short sentence:

```
from mlflow.deployments import get_deploy_client

deploy_client = get_deploy_client("databricks")

embeddings = deploy_client.predict(endpoint="databricks-bge-large-en", inputs={"input": ["What is ChatGPT?"]})
pprint(embeddings)
```

```
{'data': [{'embedding': [0.03155517578125, -0.0080108642578125, -0.01099395751953125,
                         0.06463623046875, -0.007534027099609375, -0.0513916015625,
                         -0.01045989990234375, 0.03021240234375, 0.01142120361328125,
                         0.0228271484375, -0.044403076171875, -0.0018405914306640625,
```

Figure 3.27 – Applying FMAPI with the BGE endpoint to convert "What is ChatGPT?" into an embedding

2. Now we scale out to apply this embedding conversion across all our chunks. Again, we create a `pandasUDF` for scalability purposes.

```
@pandas_udf("array<float>")
def get_embedding(contents: pd.Series) -> pd.Series:
    import mlflow.deployments
    deploy_client = mlflow.deployments.get_deploy_client("databricks")
    def get_embeddings(batch):
        response = deploy_client.predict(endpoint="databricks-bge-large-en", inputs={"input": batch})
        return [e['embedding'] for e in response.data]

    max_batch_size = 150
    batches = [contents.iloc[i:i + max_batch_size] for i in range(0, len(contents), max_batch_size)]
    all_embeddings = []
    for batch in batches:
        all_embeddings += get_embeddings(batch.tolist())

    return pd.Series(all_embeddings)
```

Figure 3.28 – PandasUDF to apply embedding conversion across all chunks

3. Apply our UDF will append our `raw_table` chunks with the corresponding embeddings.
4. Now that our data preparation process is completed, we save our table in the initially pre-created Delta table `pdf_documentation_text` using append mode.

We've ingested our PDFs for this project in one big batch, which works perfectly fine as an example. However, it also means that anytime you want to add a new PDF to your chatbot's knowledge base, you must manually rerun all the above steps. We recommend a workflow for production-grade solutions to automate the steps above and incrementally ingest PDFs as they arrive in storage.

We now have a dataset ready for a Vector Search index, which we'll cover in *Chapter 4*.